

Pipes in the Desert

Summary

E5 - TEAM A

Supervisor:
Dr. Balla Katalin

Members:

Muhammad Ibrahim Shoeb
Arda Gecegörür
İlgin Tunç
Muhammad Hameez Khan
Yahya Akhrikhar
Aasif Mohd

OZLVV3
EKQNAK
TDQWFF
TFBB32
K20UXP
OI6VE6

muhammadibrahimshoeb@gmail.com
ardagecegorur@gmail.com
ilgintunc11@gmail.com
hameezkhan993@gmail.com
yahyaakhrikhar02@gmail.com
aasifuk1122@gmail.com

14. Summary

14.1 Experiences of the project

What we learned

During this project, we learned the difference between just coding a program and doing software engineering. At the beginning, the task looked like a game implementation problem, but the semester showed us that the important part was the full engineering process: understanding the problem, writing requirements, creating analysis models, planning the skeleton and prototype, implementing the program step by step, testing it, and checking traceability.

Concretely, we learned how to translate a problem description into use cases, classes, sequence diagrams, state diagrams, test cases, and finally working Java code. The pipe system made this very visible because every rule had to appear somewhere in the design and implementation: movement, water flow, leaking pipes, broken pumps, pump directions, generated pipes and pumps, scoring, and game ending.

Generally, we learned that planning before coding saves time later. In the beginning it was tempting to start implementing immediately, but the skeleton and prototype stages helped us build the program in smaller, controllable steps. We also learned team management: dividing work, reviewing each other's parts, keeping documentation consistent, and making common decisions when the problem statement allowed different interpretations. Pair programming and working together physically also helped us find mistakes faster and agree on implementation choices.

Most difficult and easiest parts

The most difficult part was keeping the documentation, design, code, and tests consistent with each other. The dynamic pipe network was also challenging because many rules interacted at the same time: pumps route water, pipes may leak or have free ends, players can only move on pipes and pumps, and the two teams have opposite goals.

The easiest part was implementing smaller actions after the model became stable. Once the main classes such as Pipe, Pump, Player, Plumber, Saboteur, Cistern, and PipeNetwork were clear, actions like repairing a pipe, puncturing a pipe, changing pump direction, or ending a turn were easier to add and test one by one.

Time and task fit

The milestone structure mostly matched the work and helped us build the project gradually. However, the transition from diagrams to executable behavior required more time than expected. Integration, testing, and keeping documents updated took significant effort, especially when a small code decision affected several earlier documents.

The points were generally fair, but some tasks required more coordination than their size first suggested. Traceability and test documentation were especially time-consuming because they required comparing many previous documents with the final code.

Suggestions and future tasks

For future projects, it would be useful to give a small example package for each milestone, especially for expected test-case format, traceability matrix format, and final summary structure. The project task itself was suitable because it contained enough domain rules to practice analysis, design, implementation, and testing, but it was still possible to customize ambiguous parts. Similar projects should keep this balance: a clear base problem, several interacting objects, and freedom to choose simpler solutions for undefined details.

Overall conclusion

The team reached a working final GUI program with documented requirements, analysis models, skeleton plans, prototype plans, GUI plans, source code, test cases, and traceability checks. The main lesson was that a software project is not only about writing code. It is about building the right program in a controlled way, where requirements, design, implementation, testing, teamwork, and documentation support each other.

14.2 Protocol

Start (date & time)	Duration (hours)	Performer(s) name	Activity description
26.05.2026. 12:00	4 hours	Muhammad Ibrahim Shoeb, Arda Gecegorur, Ilgin Tunc, Muhammad Hameez Khan, Yahya Akhrikhar, Aasif Mohd	Meeting at BME I Building. The team worked together physically to finalize the summary documentation. Each member explained their personal learnings from the semester. As a common decision, the team agreed that the summary should emphasize the difference between coding and software engineering, the importance of building the program step by step, team management, pair work, traceability between documents and code, and the difficulty of keeping the dynamic pipe network consistent. The team also agreed to mention that clearer examples for test cases and traceability reports would help future teams.